

AccessAI

An AI-Powered Smart Accessibility Doorbell for Blind and Deaf Users

Detailed Project Report

*“Rahul is at the front door. He is carrying a parcel.
Likely a delivery. They said: ‘Package for you.’”*

Architecture · Workflow · Technologies · Algorithms · Evaluation

System status	Complete through Phase 17 of 17
Implementation	≈ 7,700 lines Python · 7,900 lines Dart · 1,500 lines web · 219 lines C++
Target platform	Python 3.12 · Linux · CPU-only · no GPU assumption
Perception models	SCRFD · ArcFace R50 · YOLOv8n · MiniFASNet · OSNet · Whisper · Kokoro
Accessible output	Blind Mode (speech + earcons) · Deaf Mode (flash + large text + vibration)
Languages	11 (English + 10 Indian languages)
Clients	Web dashboard · installable PWA · native Flutter app

Contents

Right-click and choose Update Field to build the table of contents.

1. Executive Summary

A doorbell is a single-bit device. It tells you that somebody is outside and nothing else. For a blind user that bit is nearly useless, because answering the door means opening it to an unidentified stranger. For a Deaf user the bit never arrives at all. AccessAI replaces that one bit with a sentence.

The output of the entire system is one spoken or displayed sentence. “*Rahul is at the front door. He is carrying a parcel. Likely a delivery. They said: ‘Package for you.’*” Every architectural decision in this report exists to make that sentence accurate, fast, and safe to trust.

The system is a three-tier appliance. A camera at the door — either an ESP32-CAM or a development webcam — feeds frames to a Python perception server on the home LAN. That server runs a thirteen-stage pipeline over each triggered frame, producing a single structured record. Three clients render that record in an accessible form: a web dashboard, an installable PWA, and a native Android app.

1.1 What the system does

- **Recognises household members** by face using ArcFace embeddings, and speaks their name instead of a description.
- **Detects and describes strangers** cautiously — an approximate age band, never a number; a gendered noun only when the model is confident.
- **Counts people correctly** even when some faces are not visible, by reconciling face detections against YOLO person boxes.
- **Rejects photo and screen attacks** with a MiniFASNet liveness check, downgrading a spoofed known face to “Unknown” rather than trusting it.
- **Reads courier labels and describes the scene** through a vision-language model, grounded on the local detectors so it cannot invent extra people.
- **Transcribes what the visitor says** on explicit user request, and translates it into any of eleven languages.
- **Remembers repeat strangers** across a 24-hour window by clothing and body appearance, and proposes enrolling anyone seen five or more times.
- **Answers follow-up questions** hands-free through a custom “Hey Access” wake word trained entirely offline.

1.2 Reported scale

Table 1. Implementation inventory, measured from the working tree.

Component	Language	Size	Role
accessai/ package	Python 3.12	≈ 7,700 lines across 19 modules	perception, orchestration, HTTP/WS server
mobile/lib/	Dart 3.12	≈ 7,900 lines across 36 files	Flutter client, Riverpod state, background alerts
web/	vanilla JS	≈ 1,500 lines	dashboard + PWA, no build step, no CDN
firmware/	C++ / Arduino	219 lines	ESP32-CAM ring button, PIR,

			MJPEG server
tests/	pytest	1,241 lines across 10 files	hardware-free logic suite
scripts/	Python	≈ 900 lines	model fetch, ONNX conversion, wake-word training

1.3 The three engineering commitments

Three rules recur in every module and account for most of the design. They are stated here because the rest of the report is, in effect, their elaboration.

Graceful degradation is a feature, not a fallback

No missing dependency, absent model file, unplugged camera, or dead network is allowed to crash the appliance or produce a stack trace. Every heavy module sits behind an `ENABLE_*` flag and an `available()` probe. A missing camera yields a blank frame and a valid event; a missing text-to-speech engine falls through three tiers to `espeak`; a dead vision-language model leaves the local detector facts intact. A doorbell that fails closed is not a doorbell.

Accessibility drives the technical choices, not the reverse

Approximate ages are spoken as bands because the estimator is only accurate to within several years. Speech is synthesised sentence-by-sentence with prefetch because a ten-second wait before the first word is unusable. Alert categories carry distinct vibration rhythms *and* distinct pitch contours, because contour alone is lost on users with pitch-perception differences. A regression test asserts that raw JSON is never read aloud.

Consent is enforced in the architecture

Pressing the doorbell records no audio whatsoever. Transcription happens only on an explicit user action. The wake word ships disabled because an always-open microphone is a decision the user must make deliberately. These are not settings buried in a menu; they are separate code paths.

2. Problem, Motivation and Scope

2.1 The accessibility gap

The doorbell has not changed structurally in a century: a button outside triggers a chime inside. That design embeds two assumptions — that the occupant can *hear* the chime, and can *see* who is outside before deciding to open the door. For blind and Deaf users one or both assumptions fail, and the failure is not marginal.

For a blind user the chime arrives but carries no identity and no intent. The available responses are all unattractive: open the door to an unknown person, call out and trust the answer, or ignore the door and miss deliveries and visitors. Each choice trades away either safety or independence.

For a Deaf user the problem starts earlier — the chime may not register at all. Even with a flashing-light alert, arriving at the door does not help: the visitor cannot be heard, and replying is difficult. An ordinary exchange such as “please leave it at the gate” is effectively impossible through a closed door.

2.2 Why existing products do not close it

Table 2. Existing categories and the specific gap each leaves open.

Product category	What it provides	Why it fails these users
Smart video doorbells (Ring, Nest, Amazon)	Camera, motion alerts, cloud recording, premium face and package detection	The entire interaction model is a video feed plus text or audio notifications — exactly the two channels unavailable to these users. Cloud-dependent and subscription-gated.
Assistive scene-description apps	Prove that AI scene description works from a phone camera	General-purpose and phone-held; not tied to the door, and they do not fuse identity, objects and speech into one doorstep event.
Deaf notification systems (flashing / vibrating bells)	Solve <i>awareness</i> — that somebody is present	Provide nothing once the user reaches the door. The communication problem is untouched.

The gap is a system that treats blind and Deaf users as the primary audience, fuses face, object, scene and speech into one contextual understanding, supports two-way doorstep communication, and runs locally without a subscription. The contribution of this project is that integration and reframing — not a new model.

2.3 Scope

In scope. A single front-door unit; recognition of a household's registered faces; detection of visitor-related objects; a natural-language scene description; rule-based intent inference; Blind and Deaf modes; on-demand two-way text communication; local event storage; and a development path from webcam to embedded camera.

Out of scope, stated deliberately. The system does not claim to determine a stranger's true intentions — it reports only what is visibly likely. It is not medical-grade identification, is not weather-hardened, and is not certified for commercial sale. Continuous 24×7 analytics, multi-building scale and night-vision performance are future work. Weapon detection and speech-emotion classification were **explicitly dropped**: the false-positive cost of telling a blind user that a visitor is armed, or is angry, far exceeds the value of being occasionally right.

A note on the honesty of this report. Sections 9 and 14 document known defects, uncalibrated thresholds and unverified claims in the current code. They are included because a report that lists only strengths is not an engineering document, and because several of these gaps are the natural next work items.

3. System Architecture

AccessAI is organised as three tiers with a deliberately thin capture layer, a single processing brain, and multiple accessible clients over one shared API.

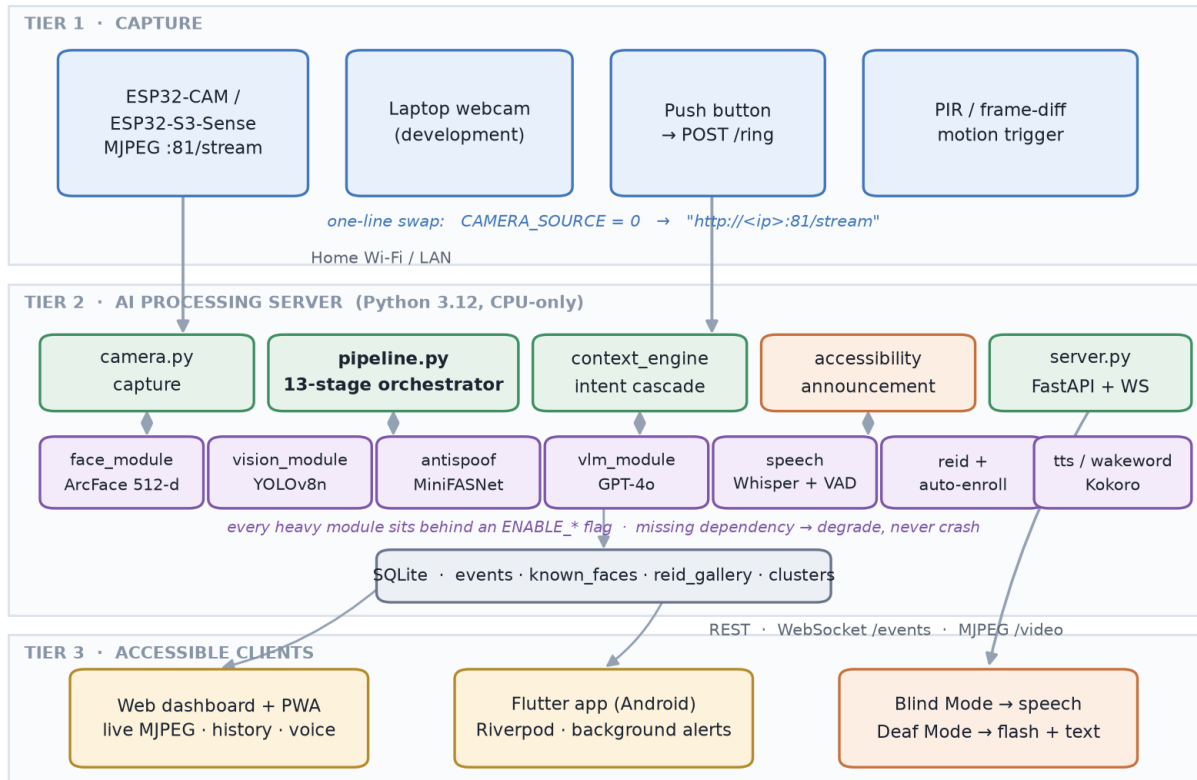


Figure 1. Three-tier architecture. The capture tier is interchangeable by configuration; all intelligence lives on the processing server; every client consumes the same REST, WebSocket and MJPEG surface.

3.1 Tier 1 — capture

The door unit is intentionally unintelligent: it captures and transmits, nothing more. This is what keeps the bill of materials near ₹2,700 and puts the compute on a machine that can actually run the models.

The whole system opens a camera in exactly one place, `accessai/camera.py`, which wraps `cv2.VideoCapture`. Because that constructor accepts either an integer index or an MJPEG URL, migrating from a laptop webcam to hardware at the door is a one-line change with nothing else affected:

```
# development - laptop webcam
CAMERA_SOURCE = 0

# deployment - ESP32-CAM MJPEG stream
CAMERA_SOURCE = "http://192.168.1.50:81/stream"
```

A background thread continuously reads frames into a single-slot buffer under a lock, so readers always receive the newest frame rather than a queued stale one. On read failure the thread reopens the source and retries with a two-second backoff — the behaviour a Wi-Fi camera requires.

3.2 Tier 2 — the processing server

A FastAPI application under uvicorn hosts the pipeline, the perception modules, the SQLite store and the client-facing API. It is CPU-only by design: every model chosen is a nano, distilled or ONNX variant, because the target is an inexpensive always-on home device, not a workstation with a GPU.

3.3 Tier 3 — accessible clients

Table 3. The three client surfaces and what each adds.

Client	Technology	Distinct capability
Web dashboard	Vanilla JS, no build step, no CDN	Live MJPEG, history, enrollment, health panel, push-to-talk
Progressive Web App	Service worker + web manifest	Installable, launches offline, cached application shell
Flutter app	Dart 3.12, Riverpod, 36 source files	Foreground service, native notification channels, on-device wake word, haptics

3.4 Design rules that hold across all modules

1. **The VisitorEvent is the spine.** Adding a capability means filling a field, never restructuring the pipeline.
2. **Central configuration.** Every tunable and feature flag lives in `config.py`; no module scatters its own constants.
3. **Graceful degradation.** A missing optional component logs a hint, reports itself unavailable, and returns empty. It never raises into the pipeline.
4. **The camera is opened in exactly one file.**
5. **Conservative language.** Announcements say “likely” and “appears to be”, never “definitely”.

4. The Data Model

Every module in AccessAI communicates through one object. This is the single most load-bearing decision in the codebase: it is why seventeen phases of features were added without ever rewriting the pipeline.

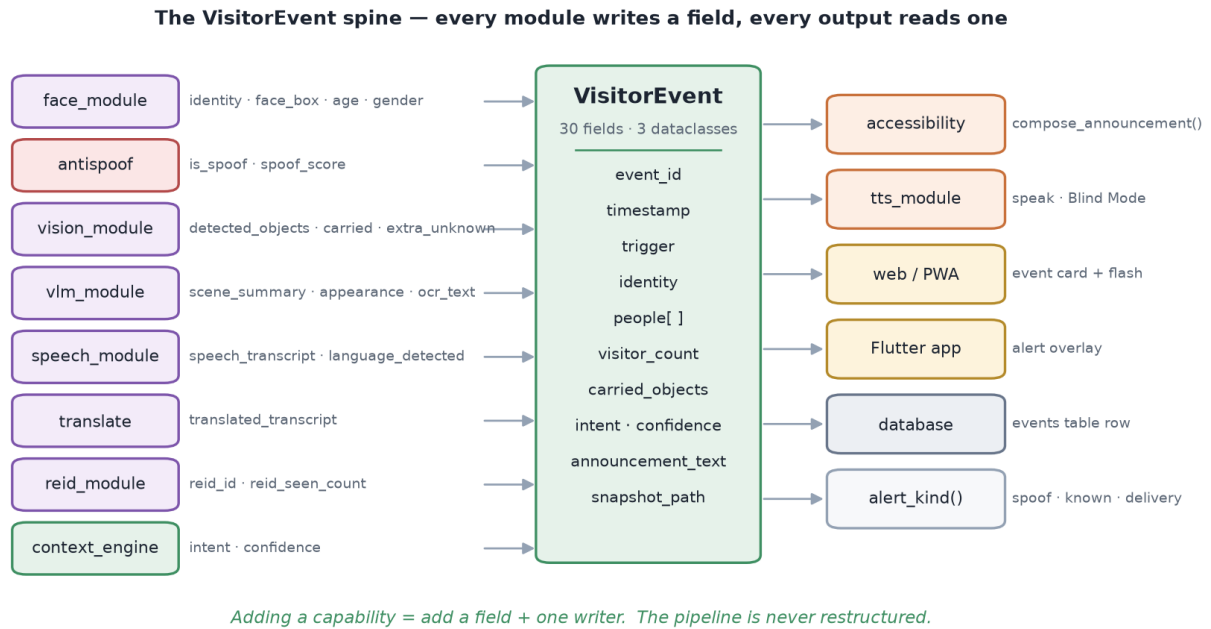


Figure 2. The VisitorEvent spine. Perception modules on the left write fields; output surfaces on the right read them. No module talks to another module directly.

4.1 VisitorEvent and its two companions

VisitorEvent is a plain dataclass of 30 fields. Only `event_id` and `timestamp` are required; everything else carries a safe default, so a partially-populated event is always valid. Two supporting dataclasses complete the model:

Table 4. The three dataclasses that make up the event model.

Dataclass	Purpose	Key fields
Identity	Event-level “who”, kept for backward compatibility with Phases 2–14	known · name · confidence
Person	One per detected face (Phase 15 multi-person support)	known · name · confidence · age · gender · box · is_spoof · spoof_score · appearance · expression
DetectedObject	One per YOLO detection	label · confidence · box

Fields are grouped by the phase that introduced them — identity and face box (Phase 2), spoof score (Phase 5), the `people` list and `extra_unknown` count (Phase 15), scene summary and OCR text (Phase 6), transcript and detected language (Phase 7), translated transcript (Phase 8), re-identification id and sighting count (Phase 9), and finally intent, confidence and announcement text.

4.2 The alert taxonomy

A pure function, `alert_kind(ev)`, reduces the whole event to one of four categories using a strict priority. It accepts either the dataclass or a dictionary rebuilt from the database, so the server, the browser and the Flutter app all derive identical results without duplicating logic.

```
1. is_spoof          -> "spoof"          # a warning, not a visit
2. identity.known OR any known -> "known"
3. "delivery" in intent -> "delivery"
4. otherwise         -> "unknown"
```

This taxonomy is what selects the earcon, the vibration rhythm, the notification channel importance and the webhook colour. Placing spoof above everything is deliberate: a photo attack is an alert about the system being deceived, not a report about a caller.

4.3 Persistence

Persistence schema — SQLite via SQLAlchemy (data/accessai.db)

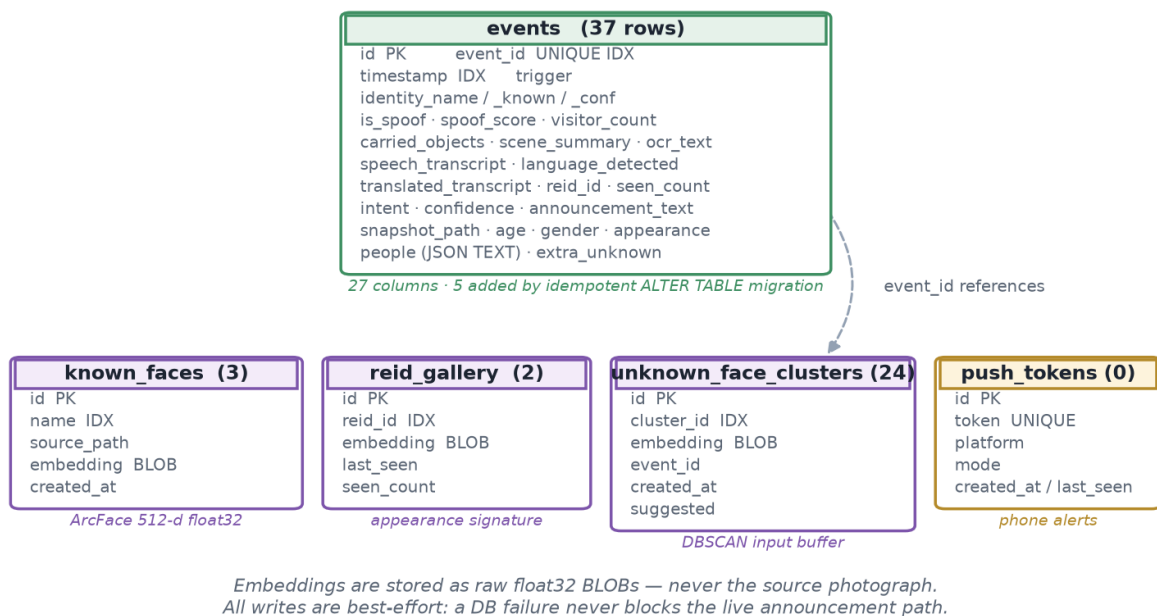


Figure 3. SQLite schema. Face and body embeddings are stored as raw float32 BLOBs; the source photograph is never written to the database.

Storage is SQLite through SQLAlchemy. Schema evolution is handled by an idempotent loop of `ALTER TABLE ADD COLUMN` statements over a small dictionary — five columns have been added this way (`age`, `gender`, `appearance`, `people`, `extra_unknown`) without a migration framework. The `people` list is serialised to a JSON text column.

Privacy by storage design. The `known_faces` and `reid_gallery` tables hold 512-dimensional float32 vectors, not images. An attacker who exfiltrates the database obtains embeddings, not photographs of the household. Every database write is best-effort inside a try/except, so a storage failure degrades history but never blocks the live announcement.

One structural asymmetry deserves recording. The in-memory face gallery is rebuilt at startup by re-running detection over the JPEGs on disk — it never reads the `known_faces` table, which

serves as a durable audit record only. The consequence is real: an embedding enrolled *without* a photo, as happens when an auto-enrollment suggestion is confirmed, does not survive a restart.

5. Development Methodology

The system was built in seventeen phases, each ending in a working and demonstrable system rather than an integration milestone. The constraint that made this possible is rule 2 of Section 3.4: every heavy feature sits behind a flag, so the base application runs at every commit even when later modules are absent or disabled.

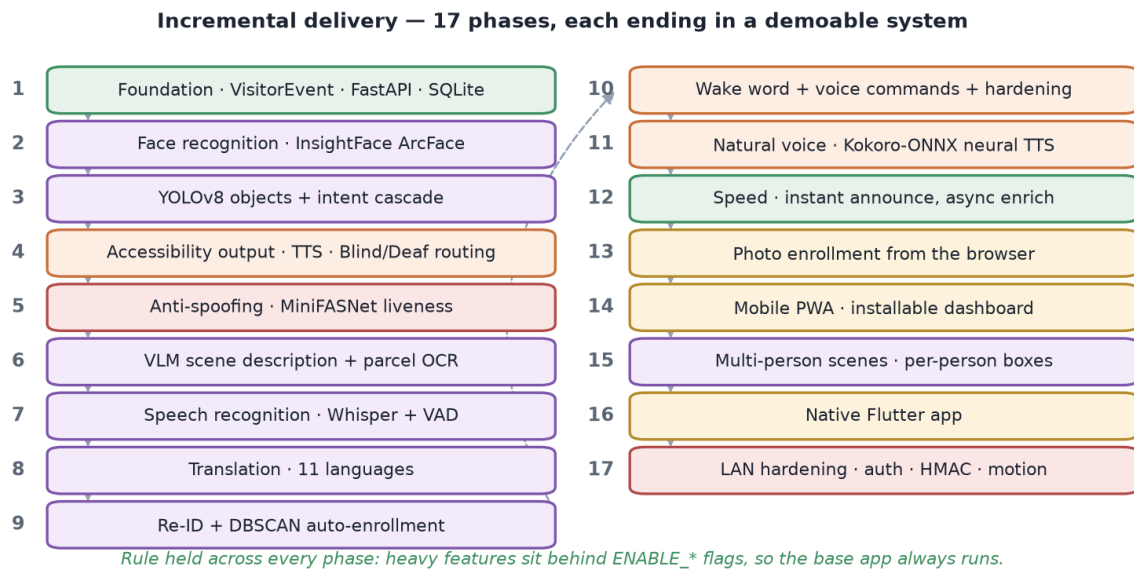


Figure 4. The seventeen-phase incremental build. Phases 1–10 established the perception pipeline and dashboard; 11–17 added natural voice, multi-person handling, native mobile and LAN hardening.

The methodology has a specific consequence for a report of this kind: because each phase was verified before the next began, the failure modes documented in Section 14 are genuinely *residual* rather than untested. Several were found by the phase verification checklists and are recorded in code comments at the site of the fix — the multi-person counting defect in Section 7.2 and the announcement connector defect in Section 11.3 are both examples.

6. The Processing Pipeline

`pipeline.py` is the core algorithm of the system. A trigger — a doorbell press, an HTTP webhook, confirmed motion, or a voice command — supplies one frame, and thirteen ordered stages transform it into a complete `VisitorEvent`.

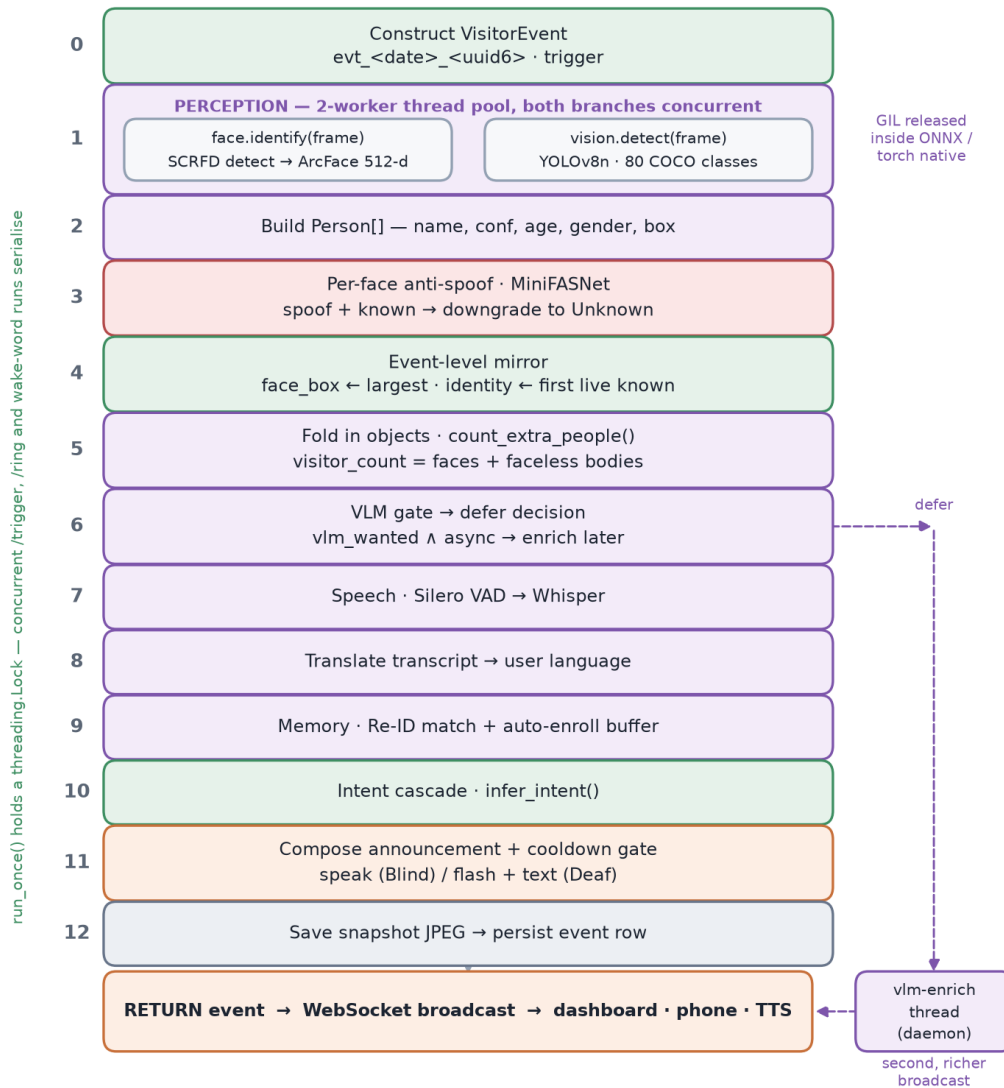


Figure 5. The thirteen pipeline stages. Stage 1 forks into two concurrent perception calls; stage 6 may defer the vision-language model to a background thread so the announcement is not delayed.

6.1 Stage ordering and why it is fixed

The order is not arbitrary. Anti-spoofing must precede the identity mirror, or a spoofed face could be promoted to the event's primary identity. Object detection must precede the visitor count, since faceless bodies contribute to it. Intent must follow everything, because it consumes spoof state, identity, carried objects and OCR text. The announcement must be last, because it reads the finished event.

Table 5. The thirteen stages of `pipeline.run_once()`.

Stage	Operation	Notable detail
0	Construct the event	<code>evt_<YYYYMMDD>_<HHMMSS>_<uuid6></code> , trigger recorded
1	Concurrent perception	Face recognition object detection on a 2-worker pool; both release the GIL inside native ONNX and torch code
2	Build the Person list	Name, confidence, age, gender and box per face
3	Per-face liveness	A confident spoof strips the name from that face only
4	Event-level mirror	<code>face_box</code> tracks the largest face; identity tracks the first live known face — deliberately different people when both are present
5	Fold in objects	Head-count reconciliation (Section 7.2)
6	Vision-language gate	Decides inline call, deferred call, or skip
7	Speech	Only when an audio array was explicitly supplied
8	Translation	Skipped when source and target language match
9	Memory	Re-identification and the auto-enrollment buffer
10	Intent	The rule cascade of Section 8
11	Announcement + cooldown	Composes, then speaks or suppresses
12	Snapshot and persist	JPEG written, then the row inserted

6.2 Concurrency and the serialisation boundary

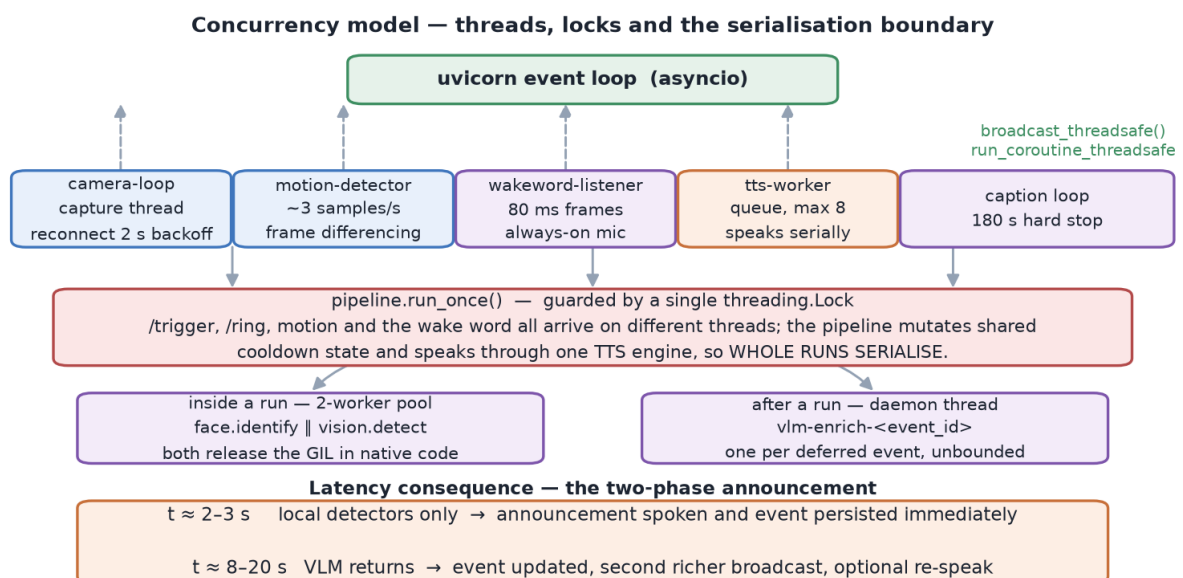


Figure 6. Thread model. Five long-lived producer threads feed the asyncio event loop, but the pipeline itself is serialised by a single lock.

Five threads run for the lifetime of the process: the camera capture loop, the motion detector, the wake-word listener, the text-to-speech worker, and — while active — the caption loop. None of them run on the asyncio event loop, so all of them publish through a thread-safe bridge that schedules a coroutine back onto the captured loop.

The pipeline itself is guarded by a single `threading.Lock`. This is a considered trade-off rather than an oversight:

```
with self._run_lock:
    return self._run_once_locked(frame_bgr, trigger, audio)
```

Requests from `/trigger`, `/ring`, the motion detector and the wake word can all arrive concurrently on different executor threads. The pipeline mutates shared cooldown state and speaks through a single text-to-speech engine, so whole runs serialise. The cost is that a second simultaneous ring waits for the first to complete; the benefit is that no lock discipline is needed anywhere inside the thirteen stages.

6.3 The two-phase announcement

The dominant latency in the pipeline is the cloud vision-language call — eight to twenty seconds against two to three for all local detectors combined. Blocking the announcement on it would make the doorbell useless.

Speed strategy. Stage 6 sets a `defer_vlm` flag. When set, the pipeline completes using local detector facts only, speaks the announcement, saves the snapshot and persists the event — then hands the frame to a background daemon thread. When the vision-language model returns, the event is updated in place and a second, richer broadcast reaches every client. The user hears “someone is at the door with a package” in about two seconds, and “wearing a blue courier uniform, the label reads BlueDart” a few seconds later.

A subtlety follows from this. The eight-second repeat-suppression cooldown is bypassed whenever the vision-language call is deferred, because the instant line is the only thing spoken at that moment — suppressing it would leave a repeat visitor hearing nothing at all until the delayed follow-up. Under the shipped configuration nearly every event defers, so the cooldown is largely inert in practice.

7. Perception Subsystems

Four independent perception modules answer four different questions about the frame. Each is separately flag-gated and separately degradable.

Table 6. The four perception modules and the question each answers.

Question	Module	Model	Runtime	Output
Who is this?	face_module	SCRFD-10GF + ArcFace R50	ONNX Runtime CPU	512-d embedding, name, age, gender
What is present?	vision_module	YOLOv8-nano	PyTorch 2.4.1 CPU	80 COCO classes with boxes
Is the face real?	antispoof	MiniFASNet V2 + V1SE	ONNX Runtime CPU	Liveness score in [0, 1]
What is happening?	vlm_module	GPT-4o (vision)	GitHub Models HTTPS	Structured JSON scene + labels

7.1 Face recognition

Recognition uses the InsightFace `buffalo_l` pack: SCRFD-10GF for detection, ArcFace ResNet-50 for the embedding, and a small gender-age network. The embedding is 512-dimensional and L2-normalised, which reduces cosine similarity to a bare dot product.

```
def _cosine_sim(a, b) -> float:
    return float(np.dot(a, b))          # both vectors are L2-normalised
```

Detections below a 0.5 detector score are discarded. The live embedding is compared against every gallery vector; the best match is accepted if similarity reaches **0.45**, otherwise the face is “Unknown”. Multiple photographs of one person produce multiple gallery entries under the same name, which measurably improves recall — this is why the enrollment UI encourages several photos.

A vision-language model is deliberately *not* used for identity. ArcFace is trained with an additive angular margin specifically so that same-identity vectors cluster and different-identity vectors separate; asking a general captioning model “who is this exact person” is the wrong tool for a one-to-one matching problem.

Person names are validated by `is_safe_person_name()` before touching the filesystem, rejecting empty strings, `.`, `...`, any `..` substring, and any path separator or null byte. This guard is applied at every enrollment, deletion and photo-retrieval entry point.

7.2 Object detection and head-count reconciliation

YOLOv8-nano runs at a 0.4 confidence threshold over the 80 COCO classes. Since COCO has no parcel class, a small mapping converts the classes that matter into spoken phrases — including book to “a package”, because nano models routinely classify small courier boxes as books.

The harder problem is counting people. A naive `person_count - face_count` produced a visible demo failure: one real visitor was announced as “multiple people at the door” because a spurious 0.47-confidence duplicate box was counted as a second person.

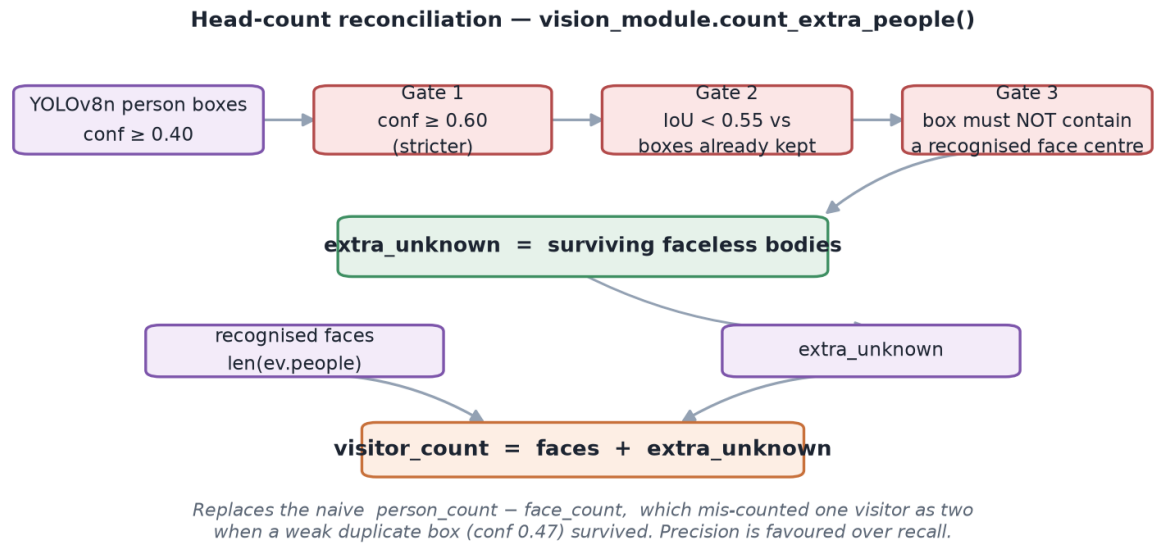


Figure 7. Head-count reconciliation. Three successive gates convert raw YOLO person boxes into genuine faceless visitors, favouring precision over recall.

The replacement applies three gates in order. A candidate person box must clear a **stricter 0.6** confidence threshold; must not overlap an already-accepted box at $\text{IoU} \geq 0.55$; and must not geometrically contain the centroid of an already-recognised face, since a body wrapping a known face is that same person. What survives is a genuinely faceless visitor, and the final count becomes recognised faces plus survivors. A visitor who has turned away still counts; a duplicate box no longer does.

7.3 Liveness detection

A printed photograph or a phone screen held to the camera must not authenticate as a household member. The anti-spoof module addresses this with two backends and a deliberately asymmetric failure policy.

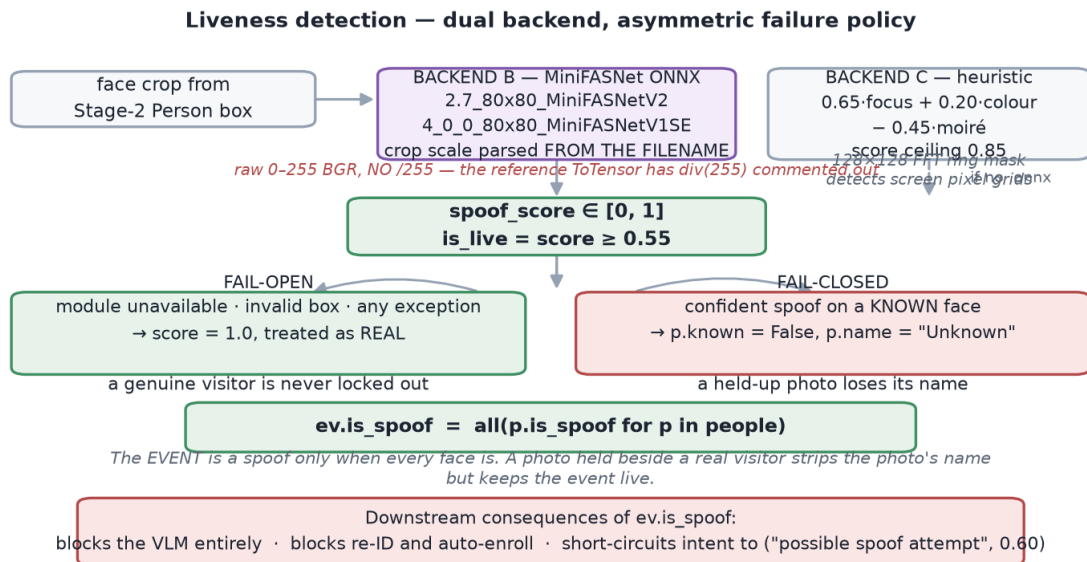


Figure 8. Liveness detection. The failure policy is asymmetric by design: fail-open when the detector is unavailable, fail-closed on a confident spoof.

The MiniFASNet path

Two MiniFASNet ONNX models run on differently-scaled crops of the same face — 2.7× and 4.0× — with the scale factor parsed from the model filename. Their softmax outputs are averaged, and class index 1 is the “real” probability. The threshold is **0.55**.

Two preprocessing details are load-bearing and counter-intuitive. The models expect raw **0–255** pixel values, because the reference implementation's `ToTensor` has its `div(255)` commented out; dividing by 255 collapses every face — genuine ones included — onto the spoof class. Channel order stays **BGR**, matching how the reference reads frames. Getting either backwards silently destroys the detector without raising an error, which is precisely the dangerous kind of failure.

The heuristic fallback

When no ONNX file is present the module falls back to a hand-built score combining three signals: Laplacian focus, colour saturation, and a moiré-pattern penalty computed from a 128×128 FFT with a radial ring mask that isolates the high-frequency band where a screen's pixel grid appears.

```

lap_var = float(cv2.Laplacian(gray, cv2.CV_64F).var())
focus  = 1.0 - np.exp(-lap_var / 55.0)
colour  = min(1.0, float(hsv[..., 1].std()) / 48.0)
screen  = self._screen_artefact_score(gray)          # FFT ring-mask moire
score   = 0.65 * focus + 0.20 * colour - 0.45 * screen

```

Worth noting for evaluation: this heuristic can never emit a score above **0.85**, so against the 0.55 threshold a face needs a Laplacian variance of roughly 42 with strong colour, or about 103 without. It is loudly logged as not production-grade.

The failure policy

Fail-open when the module is unavailable, the box is degenerate, or any exception fires: the score defaults to 1.0 and the visitor is treated as real. A genuine caller must never be locked out by a missing model file. **Fail-closed** on a confident spoof against a known face: that person's name is stripped and they become “Unknown”, while the confidence value is retained for the record.

Crucially, the event-level flag is `all(p.is_spoof for p in people)` — the whole event counts as a spoof only when *every* face is one. A photograph held up beside a real visitor loses its name while the real person stays correctly recognised. That flag then blocks the vision-language call, blocks re-identification and auto-enrollment, and short-circuits intent to “possible spoof attempt”.

7.4 Scene description and OCR

For richer description the system calls GPT-4o through the GitHub Models OpenAI-compatible endpoint. One request returns both the scene description and any visible label text, so parcel OCR costs no second round-trip — the `ocr_module` is a thin view over the same response rather than a separate engine.

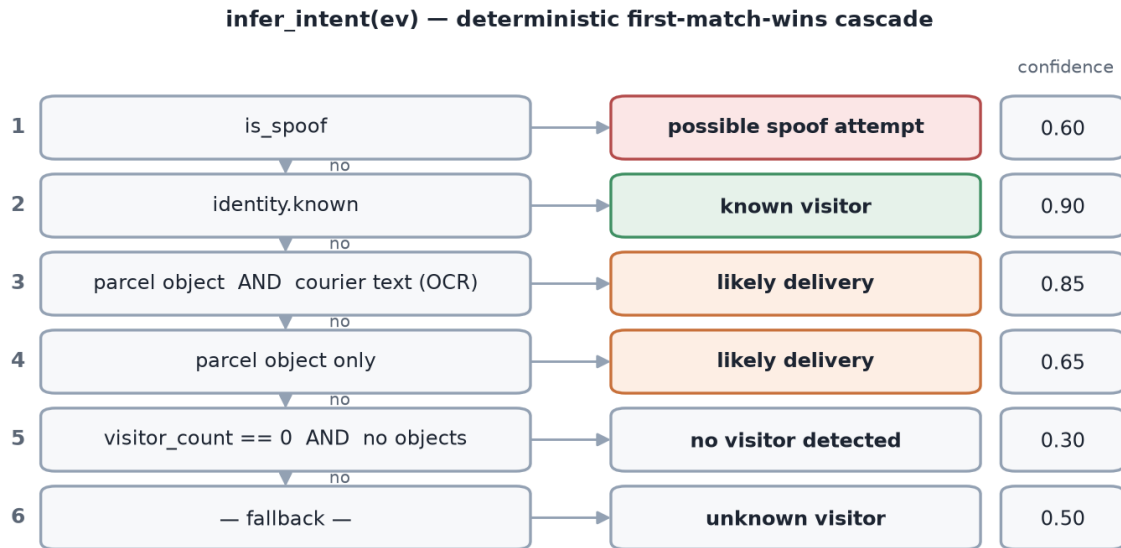
The system prompt frames the model as “the eyes of a blind person” and imposes hard rules: never state identity, exact age, gender or race; never attribute emotion or intent as fact; never produce judgements such as “suspicious”. The user prompt demands strict JSON with people ordered left-to-right.

Grounding against hallucination. The local detectors are far more reliable at counting than the vision model, which invents extra people when left unconstrained. Every request is therefore prefixed with a ground-truth preamble — for example “*2 people (known: Alex; 1 unknown); 1 of them has no clearly visible face*” — and instructed to trust it over its own count. Per-person attribution is applied only when the model's person count exactly matches the local face count, because a blind zip previously assigned one person's clothing to another.

Four layered fallbacks parse the reply, because the result is spoken aloud: strip markdown fences; extract the substring between the first { and last }; treat a brace-free reply as plain prose; and finally salvage individual fields by regular expression when the JSON is truncated. Multiple API keys are supported with rotation starting from the last known-good index, and keys are masked to their final four characters in every log line.

8. The Context Engine

Fusing the perception signals into a stated purpose is done by a deterministic rule cascade — not a classifier. The module is pure: no I/O, no configuration import, keywords injected as parameters, and fully unit-tested.



No training data, no classifier — the cascade is auditable: every announcement traces to one branch. Courier text alone never yields "delivery" (read only inside the parcel branch).

Figure 9. The intent cascade. First match wins; every announcement traces to exactly one branch.

8.1 Why rules rather than a model

- **No labelled training data exists** for doorstep intent, and manufacturing it would encode the author's assumptions rather than discover ground truth.
- **Explainability is a safety property here.** When the system tells a blind user “likely a delivery”, it must be possible to say exactly which evidence produced that claim. A cascade of six rules is auditable; a learned model is not.
- **Determinism aids testing.** The confidence values are pinned by assertions, so an accidental reordering of the branches fails the suite.

8.2 The rules and their evidence

Table 7. The intent cascade with its exact confidence values.

#	Condition	Intent	Confidence	Rationale
1	is_spoof	possible spoof attempt	0.60	A warning outranks any visit classification
2	identity.known	known visitor	0.90	Strongest available evidence — a matched biometric
3	Parcel object and courier text	likely delivery	0.85	Two independent signals agree
4	Parcel object only	likely delivery	0.65	One signal; confidence reduced accordingly

5	No people and no objects	no visitor detected	0.30	The bell rang but the frame is empty
6	Fallback	unknown visitor	0.50	Honest default

A courier keyword read by OCR *alone* never produces a delivery intent — the text signal is only consulted inside the parcel-object branch. An Amazon label visible with no bag or box detected therefore falls through to “unknown visitor” at 0.50. This is conservative by construction: text on a wall or a passing van should not manufacture a delivery.

Fourteen courier keywords are matched case-insensitively, spanning international and Indian carriers — FedEx, DHL, UPS, Amazon, USPS, BlueDart, Delhivery, DTDC, Ekart, Shiprocket — plus the generic terms *courier*, *parcel*, *package* and *delivery*.

9. Visitor Memory

Two independent mechanisms give the system memory across events: appearance-based re-identification for the current day, and face clustering for long-term enrollment suggestions. Both run only for unknown, non-spoofed visitors.

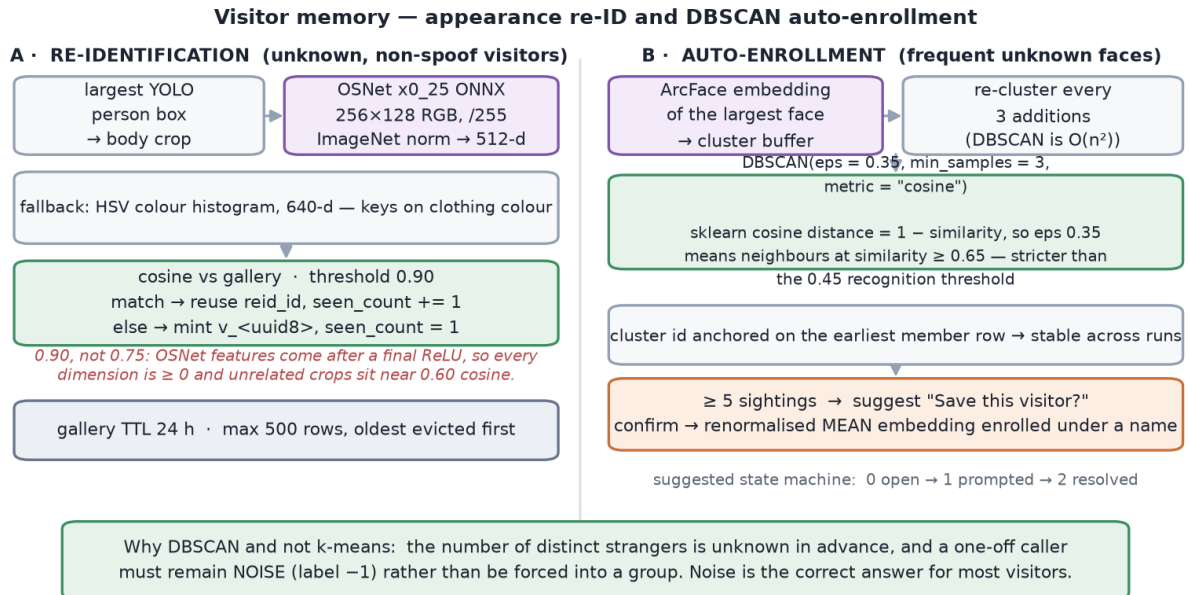


Figure 10. The two memory subsystems. Re-identification answers “have I seen this stranger today?”; auto-enrollment answers “should this stranger become a known person?”

9.1 Re-identification

When a stranger's face is not visible or not matched, the system falls back to appearance. An OSNet x0_25 network converts the largest YOLO person box into a 512-dimensional descriptor, and cosine similarity against a time-limited gallery decides whether this is a returning individual. A match increments a sighting count; a miss mints a fresh identifier. The gallery has a 24-hour time-to-live and a 500-row cap with oldest-first eviction.

Preprocessing here is the exact opposite of the anti-spoof path, and the contrast is instructive: OSNet takes **RGB**, divided by 255, with ImageNet mean and standard deviation normalisation. Two ONNX models in the same process, with two incompatible input conventions, both silently wrong if swapped.

A threshold that had to be recalibrated — and still is not validated. The match threshold is **0.90**, not the 0.75 one might expect. OSNet emits features after a final ReLU, so every dimension is non-negative and two *unrelated* crops already sit near 0.60 cosine by construction. On 33 development snapshots, unrelated people scored a median of 0.59 and a maximum of 0.91 — 0.75 would have merged most strangers into one identity. The configuration file records honestly that this is **not yet calibrated on real doorway footage**, and that measured AUC on the development webcam's upper-torso crops is approximately 0.45, i.e. chance, because OSNet expects roughly 2:1 full-body crops.

9.2 Auto-enrollment by clustering

Separately, the ArcFace embedding of every unrecognised face is buffered. Periodically — every three additions, since the algorithm is quadratic — DBSCAN clusters the buffer.

```
labels = DBSCAN(eps=0.35, min_samples=3,  
                metric="cosine").fit_predict(X)
```

DBSCAN is the correct family here rather than k-means, for a reason that is worth stating precisely: **the number of distinct strangers is unknown in advance, and most strangers should remain unclustered**. A one-off caller must stay labelled as noise rather than be forced into the nearest group, which is exactly what k-means would do.

Note that scikit-learn's cosine metric is a *distance*, equal to one minus similarity. An eps of 0.35 therefore means neighbours must reach similarity 0.65 — materially stricter than the 0.45 recognition threshold, which is appropriate: the bar for proposing to permanently save someone should be higher than the bar for recognising them.

Once a cluster reaches five sightings the user is prompted. Confirming enrolls the **renormalised mean** of the cluster's embeddings rather than any single frame, which is measurably more stable. Cluster identifiers are anchored on the earliest member row, so they remain stable across re-clustering runs even as membership grows.

10. Speech and Audio

Audio is where the accessibility requirements bite hardest, and where the most unusual engineering in the project sits.

10.1 Speech recognition, and the consent boundary

Transcription uses OpenAI Whisper (base, 74 M parameters, offline) preceded by Silero voice activity detection. The VAD gate is not an optimisation — Whisper reliably *hallucinates* text from silence and noise, so a clip that contains no detected speech must never reach it. Audio is captured at 16 kHz mono float32; a clip yielding under 0.3 s of detected speech is discarded, and the RMS energy gate that serves as fallback requires 0.005 amplitude.

Consent is structural, not configurable. A doorbell press records no audio at all — POST /trigger passes `audio=None` unconditionally. Transcription happens only through a separate, explicit user action. This is why the pipeline accepts audio as an argument rather than fetching it: the microphone is not the pipeline's to open.

10.2 Speech synthesis

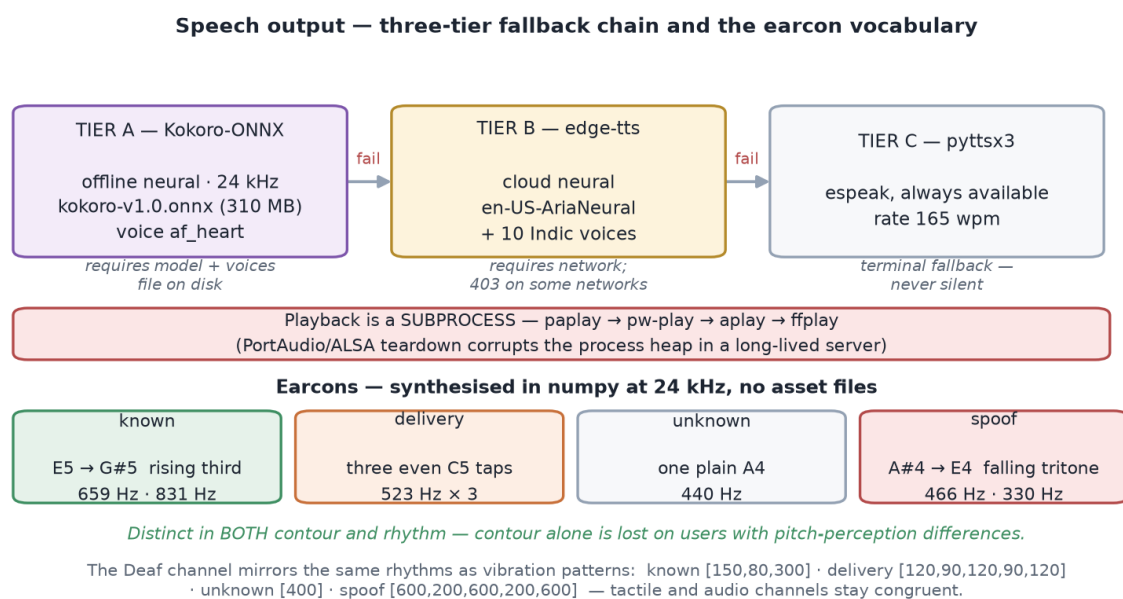


Figure 11. The three-tier synthesis fallback and the earcon vocabulary. Playback is delegated to a subprocess for process-stability reasons.

Three engines are tried in order: Kokoro-ONNX (offline neural, 24 kHz, ~310 MB), edge-tts (cloud neural, which returns HTTP 403 on some networks), and pyttsx3 driving system espeak. The active tier is probed at boot and reported through the health endpoint, so the dashboard always shows which engine is actually speaking.

Two implementation choices are worth recording. First, **playback is a subprocess** — paplay, then pw-play, aplay, ffplay — rather than an in-process audio library, because repeated PortAudio and ALSA stream teardown corrupts the heap of a long-lived server process. Second,

long announcements are **split into sentences**, with the next sentence synthesised while the current one plays, so the user hears the first words in roughly a second instead of waiting for the whole paragraph.

Earcons

Four short tone signatures are synthesised directly in numpy — no asset files ship with the project. Each alert category is distinguished in **both pitch contour and rhythm**, because contour alone is not perceivable by users with pitch-perception differences. The same rhythmic patterns are reused as vibration timings on the phone, keeping the audio and tactile channels congruent.

10.3 The offline wake word — the project's most novel component

Hands-free operation requires a wake phrase. Rather than settle for the pretrained “hey jarvis” model, a custom “Hey Access” detector was trained **entirely offline, with no human recordings and no network access**, by using the project's own text-to-speech engine as the data source.

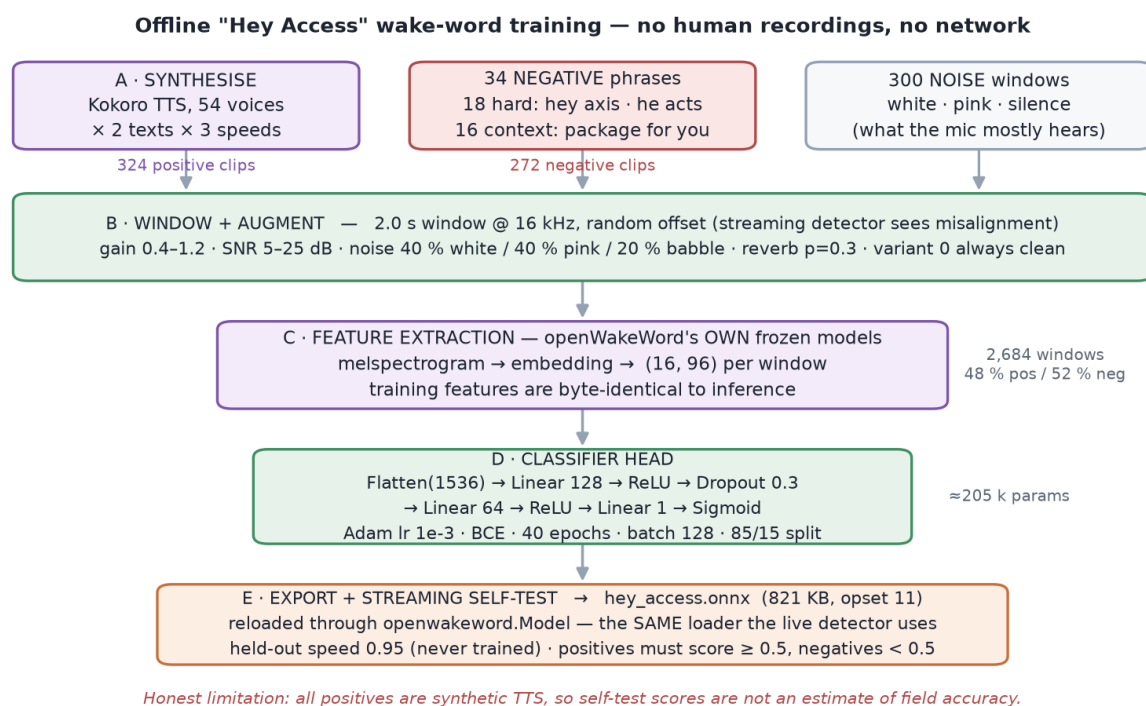


Figure 12. The offline wake-word training pipeline. Kokoro synthesises the corpus, openWakeWord's own frozen feature models extract the representation, and only a small classification head is trained.

The pipeline synthesises 324 positive clips by sweeping 54 Kokoro voices across two phrasings and three speaking rates, and 272 negatives from 34 phrases split between **hard negatives** that are acoustically adjacent (“hey axis”, “he acts”, “hey access point”) and contextual negatives drawn from the actual deployment domain (“package for you”, “who is at the door”). A further 300 pure-noise windows represent what the microphone hears the overwhelming majority of the time.

Two design points make the result usable rather than merely trainable:

6. **Features are extracted with openWakeWord's own frozen melspectrogram and embedding models**, so the training representation is byte-identical to what the live detector computes. Only a small head is learned — flatten 1536, dense 128, dropout 0.3, dense 64, dense 1, sigmoid — about 205 k parameters, trained for 40 epochs with Adam.
7. **Windows are cut at a random offset** within each clip rather than aligned to its start, because a streaming detector never sees a conveniently-aligned phrase. Augmentation applies random gain, 5–25 dB SNR noise, and probabilistic reverb, with the first variant of each clip always kept clean.

The exported 821 KB ONNX model is verified by reloading it through the real `openwakeword.Model` loader and streaming held-out clips at a speaking rate never seen in training. Both the artefact and the loader path are therefore the production ones.

Honest limitation. Every positive example is synthetic speech from a single TTS family. The self-test confirms the model works end-to-end in the streaming loader, but it is *not* an estimate of accuracy against real human voices, accents or room acoustics. Measuring that requires human recordings the project does not have.

The detector ships **disabled**. An always-open microphone is a privacy decision that belongs to the user, and it is exposed as an explicit runtime toggle rather than a default.

10.4 Voice commands

Command handling is split into a pure function and an effectful one — a division that exists specifically to make the logic testable without hardware. `parse_command()` maps a transcript to one of seven intents using ordered keyword matching and touches nothing; `handle_command()` is the only function that reaches the world. Both push-to-talk and the wake word share the identical glue, so the two entry points cannot drift apart.

Table 8. The voice command grammar.

Utterance	Intent	Action
“who is at the door?”	who_is_there	Runs the pipeline, speaks the result
“what do you see”	analyze_now	Same, phrased as a fresh analysis
“recent visitors”	recent	Summarises the last stored event
“how many visitors today”	count_today	Counts today's events
“open the camera”	open_camera	Focuses live view on the dashboard
“blind / deaf / both mode”	set_mode	Switches accessibility mode
anything else	unknown	A spoken, helpful fallback

10.5 Multilingual output

Eleven languages are supported: English plus Hindi, Bengali, Telugu, Marathi, Tamil, Gujarati, Kannada, Malayalam, Punjabi and Urdu. Translation runs through the same GitHub Models endpoint as the vision model, and is skipped entirely when the source and target languages match. The synthesis layer selects a locale-appropriate voice for the chosen language.

11. The Accessibility Layer

This is the module the user actually experiences. Everything upstream exists to fill the fields it reads.

11.1 Composing the announcement

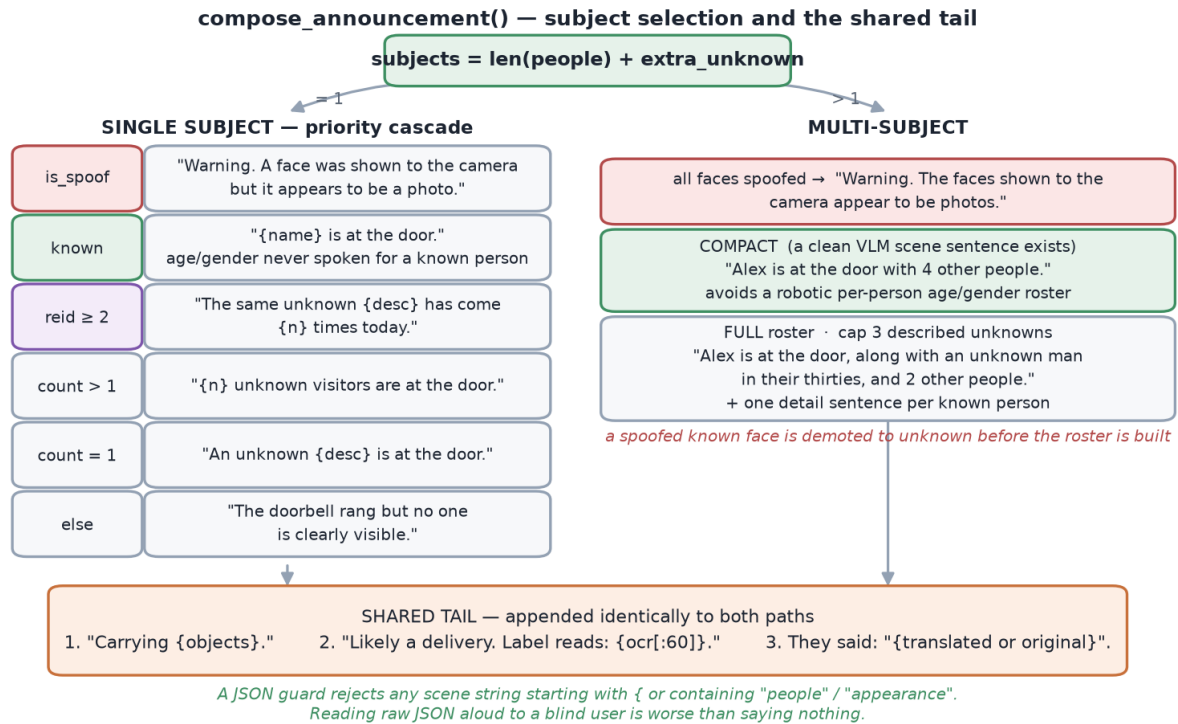


Figure 13. Announcement composition. A subject count selects one of two paths; both converge on an identical tail.

The number of subjects is `len(people) + extra_unknown`, which is why the counting work of Section 7.2 matters so directly — an inflated count changes the sentence a user hears from “Alex is at the door” to “multiple people are at the door”.

For a single subject a six-branch priority cascade selects the opening sentence, with spoof warnings outranking everything. For multiple subjects the module prefers a **compact** phrasing built on the vision model's scene sentence — “Alex is at the door with four other people” — falling back to a roster capped at three described strangers. The cap exists because a robotic recitation of every person's estimated age and gender is worse than a summary.

11.2 Rules that protect the user

- **Age is spoken only as a band** — “in their thirties” — never as a number, because the estimator is not accurate to the year and a spurious “34” invites false confidence.
- **Age and gender are never spoken for a known person.** Once a name is available, demographic guesswork adds nothing and can offend.
- **A JSON guard** rejects any scene string beginning with { or containing the tokens `people` or `appearance`. Reading a raw model response aloud to a blind user is worse than silence, and a regression test enforces this.

- **Gendered nouns require confidence.** Below the threshold the neutral “person” is used rather than a guess.

11.3 A defect worth documenting

An earlier version emitted “Alex is at the door, and 2 other people.” The connector was wrong whenever the roster contained no described strangers — a grammatical error, but one that reads as a system malfunction to someone who only ever hears the output. The fix selects the connector from the roster's actual contents rather than assuming it is non-empty. It is recorded here because it illustrates a general property of this project: for a speech-first interface, **phrasing bugs are functional bugs.**

11.4 Mode routing

Table 9. Output routing by accessibility mode.

Mode	Channel	Behaviour
Blind	Audio	Earcon, then the announcement spoken sentence-by-sentence with prefetch; hands-free follow-up questions
Deaf	Visual + haptic	Full-screen flash, large-text card, category-specific vibration rhythm, two-way typed reply spoken at the door
Both	All channels	Audio and visual paths run together

Two-way communication closes the loop that ordinary doorbells leave open for Deaf users: the visitor's speech becomes text on the user's screen, and text the user types is synthesised and played at the door. Modes can additionally be overridden per device, so a Deaf user's phone and a shared household tablet need not agree.

12. Server, Clients and Hardware

12.1 The HTTP and WebSocket surface

FastAPI exposes roughly forty endpoints. The selection below covers the ones that carry the system's actual behaviour.

Table 10. Principal API surface.

Method	Path	Purpose
GET	/video	MJPEG live stream (multipart boundary frames)
POST	/trigger	Run the pipeline on the current frame — the doorbell
POST	/ring	Hardware webhook; accepts a posted JPEG or uses the latest frame
GET	/status	Central health: every module, flags, torch version, TTS tier
POST	/listen	Push-to-talk voice command
POST	/hear_visitor	Explicit, consent-gated visitor transcription
GET	/history, /event/{id}	Stored events and snapshots
POST	/enroll, /reply, /mode	Face enrollment, two-way reply, mode switch
GET	/suggestions	Pending auto-enrollment prompts
WS	/events	Live event and voice broadcast to all clients

The health endpoint deserves specific mention as an engineering artefact. Every module reports one of `ok`, `placeholder`, `unavailable` or `off`, and the same table is printed as a self-check at boot. Because the system degrades rather than crashes, an explicit and honest health surface is the only way to know what is actually running.

12.2 Security hardening

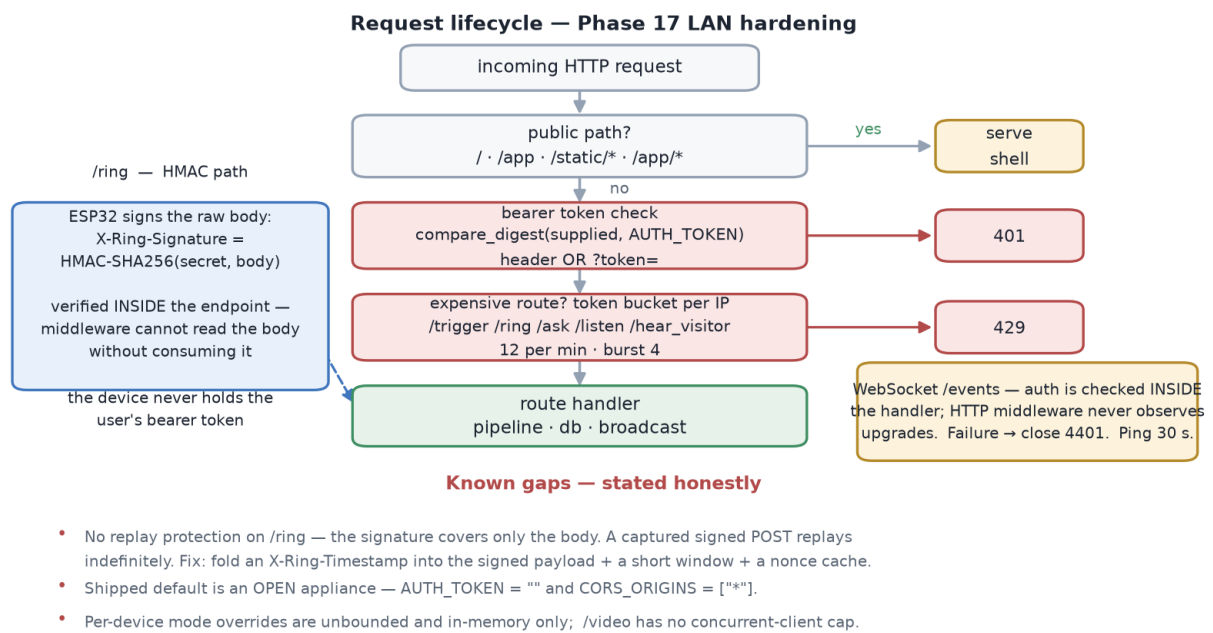


Figure 14. Request lifecycle under Phase 17 hardening, with the known gaps stated.

Three mechanisms were added for LAN deployment. Bearer-token authentication uses a constant-time comparison and accepts the token in a query parameter as well as a header, because neither an MJPEG `<img>` tag nor a browser WebSocket can set headers. A per-IP token bucket — twelve requests per minute with a burst of four, on a monotonic clock — protects the expensive pipeline routes. And POST `/ring` accepts an HMAC-SHA256 signature over the raw body, so the door hardware can authenticate without ever holding the user's bearer token.

Gaps that remain open. The `/ring` signature covers only the body, with no timestamp or nonce, so a captured signed request replays indefinitely; the fix is to fold a timestamp into the signed payload and keep a short nonce cache. The shipped defaults are also an *open* appliance — empty auth token and wildcard CORS — which suits a trusted LAN and does not suit anything else.

12.3 The Flutter client

The Android application is 7,900 lines of Dart across 36 files, using Riverpod for state management. Beyond mirroring the dashboard it adds capabilities the browser cannot offer: a foreground service that keeps alert polling alive when the app is backgrounded, native notification channels whose importance is derived from the alert category, on-device wake-word listening, and category-specific vibration patterns matching the earcon rhythms.

12.4 Progressive web app

The dashboard is installable. A service worker caches the application shell so the app launches without a network, while live data paths stay network-first. The web tier uses no build step, no

bundler and no CDN — an appliance on a home LAN should not depend on external asset delivery to render its own interface.

12.5 Door hardware

The ESP32-CAM firmware is 219 lines of Arduino C++. It serves the standard MJPEG stream on port 81, debounces a momentary button on a GPIO, and issues `POST /ring` on a falling edge — optionally attaching its own JPEG capture, in which case the server reports the frame source as `posted-jpeg` rather than `latest-frame`.

Table 11. Bill of materials for the door unit, within the ₹5,000 project budget.

Component	Purpose	Approx. ₹
ESP32-S3-Sense (XIAO) — camera + microphone	Capture image and audio	1,000–1,200
Amplified I2S speaker	Play replies at the door	300
PIR motion sensor	Wake on approach	100
Push button, status LED, wiring	Doorbell and indicator	200
18650 battery and charging circuit	Power	600
Enclosure	Weather housing	300
Laptop (already owned)	AI processing server	0
Prototype total		≈ 2,700

The ESP32-S3-Sense is recommended over the cheaper AI-Thinker board for two practical reasons: it has an on-board microphone, and it flashes over native USB-C without the GPIO0 grounding procedure the AI-Thinker requires.

13. Testing and Evaluation

13.1 What is tested, and how hardware is avoided

The suite is 1,241 lines across ten files and touches no camera, microphone, network or speaker. This is achieved by the pure-function split described throughout: the logic worth testing was deliberately separated from the I/O that would make testing hard.

Table 12. Test coverage by file.

Test file	Covers
test_context_engine.py	Intent branch priority and exact confidence values
test_accessibility.py	Announcement composition for every branch and add-on
test_voice_commands.py	Intent parsing, plus the glue driven with fakes
test_reid.py	Cosine match — reuse versus mint — on a temporary gallery
test_auto_enroll.py	DBSCAN clustering to suggestion; noise stays unclustered
test_motion.py	Frame-difference triggering and debounce
test_database.py	Schema creation, migration idempotency, round-trips
test_vlm_parse.py	All four JSON-recovery fallback layers
test_wakeword.py	Detector lifecycle and threshold handling

13.2 Measured characteristics

Table 13. Key operating parameters and where each number comes from.

Property	Value	Basis
Instant announcement latency	≈ 2–3 s	Local detectors only, CPU
Enriched follow-up latency	≈ 8–20 s	Cloud vision-language round-trip
Face embedding	512-d, threshold 0.45	ArcFace R50, L2-normalised
Object detection	80 classes, threshold 0.4	YOLOv8n, ~6 MB
Extra-person gate	0.6 confidence, IoU 0.55	Tuned to fix a demo failure
Liveness threshold	0.55	Averaged MiniFASNet softmax
Re-ID threshold	0.90	Empirical, 33 dev snapshots
Wake-word threshold	0.5	openWakeWord score
Stored events in development DB	37 with snapshots	Working tree

13.3 What has not been measured

Scientific honesty requires separating what was verified from what was merely built.

- **No accuracy figures against a labelled dataset.** Face recognition, liveness and re-identification thresholds were tuned on development footage from one webcam, one location and a handful of subjects. No precision, recall or ROC figures are claimed.
- **Re-identification is known to underperform** on the development camera's upper-torso crops, with a measured AUC near chance, because OSNet expects approximately 2:1 full-body crops.

- **Wake-word accuracy against real speech is unmeasured**, since all training positives were synthetic.
- **No evaluation with blind or Deaf users has been carried out.** For an accessibility project this is the most significant gap, and it is the first item of future work.

14. Limitations, Ethics and Future Work

14.1 Technical limitations

- Intent is inferred from visible cues and can be wrong; it is a hint, not a guarantee.
- Accuracy degrades in poor lighting, and an embedded camera is optically weaker than a laptop webcam.
- The vision-language model can mis-describe a scene, which is why local detector facts are retained and used to ground it.
- Running several models adds latency, mitigated by analysing one triggered snapshot rather than continuous video, and by the two-phase announcement.
- The in-memory face gallery rebuilds from photographs on disk, so an embedding enrolled without a photograph does not survive a restart.
- The `/ring` HMAC has no replay protection.

14.2 Ethics and privacy

Because the system combines cameras, microphones and face recognition, privacy was treated as a design constraint rather than a policy statement appended at the end.

8. **Local-first processing.** Face recognition, object detection, liveness, re-identification, transcription and synthesis all run on the home machine. Only the optional scene description leaves the network, and the system runs fully without it.
9. **Embeddings, not photographs,** are stored in the database.
10. **Trigger-based capture,** not continuous surveillance.
11. **Explicit uncertainty.** The system says “likely”, and never converts a probability into a fact the user might act on.
12. **Consent for audio is structural.** A doorbell press records nothing; the wake word ships off.
13. **Refusals encoded in the prompt.** The vision model is instructed never to infer identity, race, emotion or suspicion — categories where a confident wrong answer causes real harm.

14.3 Future work

In priority order, reflecting the gaps this report has documented:

14. **Evaluation with blind and Deaf users.** Everything else is secondary; the announcement wording and alert design are hypotheses until tested with the people they are for.
15. **Calibrate re-identification on real doorway footage** with full-body crops, and report a proper ROC rather than a hand-tuned threshold.
16. **Validate the wake word against human speech,** including accented English.
17. **Close the replay gap** on the hardware webhook.
18. **Persist enrolled embeddings independently of photographs,** removing the restart asymmetry.
19. **Multi-camera and multi-door support** through the existing event-driven architecture.
20. **On-device edge deployment** on Jetson-class hardware for a fully offline, subscription-free appliance.

15. Conclusion

AccessAI reframes the most familiar object in the home as an accessibility instrument. By combining face recognition, object detection, liveness verification, scene understanding, speech recognition and synthesis — and fusing them through a deliberately conservative context engine into a single `VisitorEvent` — it converts a meaningless chime into actionable understanding: who is at the door, what they carry, why they are likely there, and what they said.

The technical contribution is not a new model. It is the accessibility-first *integration* of proven components into a system that degrades gracefully, states its uncertainty honestly, and communicates through whichever sense the user actually has. Several individual pieces are nonetheless novel in their own right: the fully offline synthetic wake-word training pipeline, the ground-truth prompt conditioning that stops a vision model inventing visitors, the three-gate head-count reconciliation, and an earcon vocabulary distinguished in both pitch and rhythm.

Equally, this report has documented what the system does not yet do: it has not been evaluated with the users it is built for, its re-identification threshold is admittedly uncalibrated, and its wake word has never been tested against a human voice. Those gaps are stated plainly because an accessibility system that overstates its reliability is worse than one that admits its limits — the whole design philosophy of the announcement layer, with its hedged language and its refusal to guess, applies equally to the document describing it.

What exists today is a complete, working, seventeen-phase system: roughly 7,700 lines of Python, 7,900 of Dart, a hardware path costing under ₹2,700, eleven languages, three client surfaces, and a test suite that runs without a camera. It is buildable from affordable components, it runs on a CPU, it keeps its data in the home — and it turns one bit of information into a sentence.

16. References and Tools

- [1] Deng, J. et al. *ArcFace: Additive Angular Margin Loss for Deep Face Recognition*. CVPR, 2019. — the recognition embedding used via InsightFace `buffalo_l`.
- [2] Guo, J. et al. *SCRFD: Sample and Computation Redistribution for Efficient Face Detection*. 2021. — the detector in the same pack.
- [3] Zhou, K. et al. *Omni-Scale Feature Learning for Person Re-Identification (OSNet)*. ICCV, 2019. — the appearance descriptor, deployed as `osnet_x0_25`.
- [4] Minivision. *Silent-Face-Anti-Spoofing (MiniFASNet)*. — the liveness models, deployed as two ONNX classifiers.
- [5] Jocher, G. et al. *Ultralytics YOLOv8*. 2023. — object detection, nano variant.
- [6] Radford, A. et al. *Robust Speech Recognition via Large-Scale Weak Supervision (Whisper)*. 2022. — transcription, base model.
- [7] Silero Team. *Silero VAD*. — neural voice activity detection.
- [8] Ester, M. et al. *A Density-Based Algorithm for Discovering Clusters (DBSCAN)*. KDD, 1996. — auto-enrollment clustering, via scikit-learn.
- [9] *openWakeWord*. — the streaming wake-word framework whose frozen feature models were reused for offline training.
- [10] *Kokoro-82M / kokoro-onnx*. — offline neural speech synthesis.
- [11] FastAPI, uvicorn, SQLAlchemy, OpenCV, ONNX Runtime, PyTorch 2.4.1, Flutter, Riverpod, Espressif ESP32 Arduino core.

Repository layout. `accessai/` perception and server · `web/` dashboard and PWA · `mobile/` Flutter client · `firmware/` ESP32 sketch · `scripts/` model fetch, ONNX conversion and wake-word training · `tests/` hardware-free suite · `docs/` phase prompts, hardware bring-up and this report.